

Modélisation et simulation distribuée du mouvement d'un banc de poissons

Master 1 - CHPS
ISTY - UVSQ - Paris Saclay
Année universitaire 2025-2026

PPN - Sujet 7

19 décembre 2025

Encadrant : Soufian BEN AMOR

Composition de l'équipe :

Iram MADANI-FOUATIH

Yohann FRONT-REIGNIER

Laurence HUANG



Table des matières

1	Introduction	2
1.1	Contexte général	2
1.2	Problématiques	2
1.3	Objectifs du projet	2
1.4	Organisation de l'équipe	2
2	Etat de l'art	3
2.1	Historique et travaux fondateurs	3
2.2	Approches existantes	3
2.3	Technologies et outils	3
3	Méthodes et architecture	4
3.1	Architecture générale	4
3.1.1	Relations entre composants	4
3.1.2	Flux de données et contrôle	4
3.1.3	Organisation modulaire et extensibilité	4
3.1.4	Pattern de conception utilisé	5
3.2	Modèle mathématique	5
3.2.1	Règle 1 : Séparation	5
3.2.2	Règle 2 : Alignement	5
3.2.3	Règle 3 : Cohésion	6
3.2.4	Paramètres clés	6
3.3	Choix de conception	6
3.3.1	Structure de données pour les boids	6
3.3.2	Détection des voisins	6
3.3.3	Gestion des bords	6
3.3.4	Séparation modulaire Boid/Flock	6
3.4	Implémentation	7
3.4.1	Structure de données	7
3.4.2	Algorithmes principaux	7
3.4.3	Gestion de la mémoire	7
3.4.4	Optimisations appliquées	7
3.4.5	Contributions par membres	7
3.5	Tests et validation	7
3.5.1	Stratégie de test	7
3.5.2	Tests unitaires par classe	8
3.5.3	Validation des comportements émergents	8
3.5.4	Converture de tests	9
3.6	Répartition des contributions	9

1 Introduction

1.1 Contexte général

Ce projet a pour but de simuler les comportements collectifs d'un banc de poissons. Ce type de simulation est notamment utilisé dans la biologie, les jeux vidéos ou bien dans les projets d'"essaims" d'avions autonomes et est intéressant dans la modélisation de systèmes multi-agents.

Ce projet s'inscrit dans le cadre du Master 1 Calcul Haute Performance et Simulation (CHPS) de l'Université de Versailles Saint-Quentin-en-Yvelines.

L'objectif du premier semestre est de développer une version séquentielle fonctionnelle et visuellement cohérente du modèle, tandis que celui du second semestre est d'optimiser cette simulation par des approches de parallélisation et d'améliorer la fidélité du comportement collectif.

1.2 Problématiques

Une des premières problématiques à laquelle nous avons dû faire face était comment simuler des comportements complexes tels que le mouvement d'un banc de poisson.

Ensuite, nous avons vite remarqué, pendant l'implémentation et la recherche d'algorithmes, que nous allions nous retrouver avec des problèmes de performances évident en fonction du nombre d'agents à simuler et de la taille du rayon de perception de ces derniers.

Enfin, un problème majeur a émergé au moment de l'implémentation des comportements étant comment savoir si notre modèle reproduit bien le comportement naturel.

Les réponses que nous avons trouvées seront détaillées plus tard dans ce rapport.

1.3 Objectifs du projet

Nous nous sommes fixés 3 objectifs principaux :

- Implémentation des règles de Reynolds
- Création d'une visualisation interactive en 2D
- Validation visuelle des comportements

Ce qui nous permet d'avoir une base solide afin de pouvoir après nous lancer dans la partie de parallélisation lors du deuxième semestre avec un code propre et fonctionnel.

Nous nous sommes aussi fixés 2 objectifs secondaires :

- Création de la visualisation en 3D
- Préparation du code pour la parallélisation

Ces objectifs secondaires nous permettent de faciliter la transition du modèle séquentiel à un modèle parallèle ainsi que d'aller plus loin dans la simulation et de pouvoir être plus proche de la réalité.

1.4 Organisation de l'équipe

Composition et contributions :

Membre	Responsabilité	Compétences clés
Yohann FRONT-REIGNIER	Architecture & Rapport, Vector2D, Documentation	C++, CMake, LaTeX, Doxygen
Iram MADANI-FOUATIH	Algorithmes Reynolds, Boid, Flock, GUI	Math, Physique, SFML, UI/UX
Laurence HUANG	Optimisation & Tests, Flock, SpatialGrid	Structures données

TABLE 1 – Répartition des rôles et compétences au sein de l'équipe

Nous avons choisi de nous séparer les tâches en fonctions des classes à implémenter, chaque membre était responsable d'un composant principal et chaque classe implémentée se devait d'être entièrement testée grâce au module GoogleTest.

Comme outils collaboratif nous avons utilisé Github car c'est l'outil que nous maîtrisons le mieux. Le code a été mis en commun en utilisant CMake et il a été testé grâce à CTest. La documentation, quand à elle, a été gérée par Doxygen.

Pour les réunions, nous en faisons toute les 2 semaines avec notre encadrant et environ toute les semaines entre nous afin de faire le point sur nos avancées personnelles.

2 Etat de l'art

2.1 Historique et travaux fondateurs

Nous nous sommes basés sur des travaux réalisés par les travaux de Craig Reynolds en 1987 "*Flocks, Herds, and Schools : A Distributed Behavioral Model*" et "*Novel Type of Phase Transition in a System of Self-Driven Particles*" de Tamas Vicsek, Andras Czirok, Eshel Ben-Jacob, Inon Cohen, and Ofer Shochet.

Les boids sont un élément clé pour cette modélisation d'un banc de poissons. En effet, le modèle des boids nous permet, avec seulement trois règles relativement simple, de simuler des comportements complexes comme celui d'un banc de poisson. Ils sont aussi utilisés dans des domaines tels que les films ou les jeux afin d'avoir des comportements réalistes.

Afin d'aller plus loin, nous pouvons modéliser les boids en 3 dimensions, ajouter des obstacles ou des prédateurs afin d'observer les différents résultats.

2.2 Approches existantes

Il existe déjà un certain nombre d'approches utilisées pour modéliser ce genre de comportements, notamment l'approche 'force-based' et l'approche 'velocity-based'.

L'approche '**force-based**', qui est celle de Reynolds, consiste en le calcul de forces de steering (séparation, alignement, cohésion) puis l'application de ces forces qui vont influencer l'accélération.

Avantage : Simulation physique réaliste, comportement émergents naturels

Inconvénient : Coût calculatoire $O(n^2)$ pour la détection de voisins

L'approche '**velocity-based**', quand à elle, fait un travail directement sur les vitesses sans passer par les forces.

Avantage : Calcul plus rapide

Inconvénient : Moins de contrôle fin sur les comportements

Nous avons donc choisi d'utiliser l'approche force-based pour notre simulation afin de pouvoir être au plus proche du réel avec une simulation physique réaliste même si elle est coûteuse, nous avons confiance en nos futures optimisations pour compenser ce problème.

2.3 Technologies et outils

Pour notre implémentation, nous avons choisi d'utiliser *C++20* afin d'avoir des performances optimales et de pouvoir, plus tard, paralléliser plus simplement, pour avoir de meilleures performances ainsi que le support de la programmation orienté objet et parce que Python n'était pas autorisé dans ce projet.

Nous avons utilisé *SFML 2.6.1* pour l'implémentation de l'interface graphique pour sa simplicité d'utilisation et la possibilité d'ajouter assez simplement des sliders. Nous allons migrer sur OpenGL lors de la mise en 3 dimensions.

Nous avons aussi utilisé le framework de test unitaire *GoogleTest* afin de pouvoir tester toutes nos méthodes implémentées.

Nous avons aussi utilisé *CMake 3.20+* afin de pouvoir gérer plus simple les dépendances et les différentes configurations.

Pour finir, nous avons commenté notre code en utilise *Doxygen* qui nous permet de généré automatiquement une documentation complète de notre projet.

3 Méthodes et architecture

3.1 Architecture générale

Nous avons décidé de faire notre simulation en suivant une architecture modulaire en couches où chaque classe à un rôle définie. La figure 1 montre les relations entre les différentes classes.

3.1.1 Relations entre composants

Dépendances :

- *Vector2D* est la classe de base sans dépendance externe
- *Boid* dépend de *Vector2D* pour représenter sa position, sa vitesse ainsi que son accélération
- *Flock* utilise une collection de *Boid*
- *SpatialGrid* est une classe imbriquée dans *Flock*
- *GUI* utilise *Flock* mais n'est pas directement dépendant de celle ci

Multiplicités :

- Un *Flock* contient 0 à N *Boid*
- Un *Boid* appartient à 1 seul *Flock*
- Un *Flock* possède 1 seul *SpatialGrid*
- Une cellule de *SpatialGrid* contient entre 0 et N pointeurs vers *Boid*

3.1.2 Flux de données et contrôle

La boucle principale de simulation suit ce cheminement :

1. Initialisation :

- Création du *Flock* avec dimensions de la fenêtre
- Population avec *populateRandom(N)*

2. À chaque frame :

- *grid.clear()* : Vide la grille spatiale
- Pour chaque boid : *grid.addBoid()* : Insertion dans les cellules correspondante
- *flock.updateAll(deltaTime)* :
 - Pour chaque boid :
 - *getNeighbors()* par *SpatialGrid*
 - Calcul des forces : *separate()*, *align()*, *cohesion()*
 - Application poids : $F_{total} = w_{sep} \cdot F_{sep} + w_{ali} \cdot F_{ali} + w_{coh} \cdot F_{coh}$
 - *applyForce(F_total)*
 - *update(deltaTime)* : Intégration d'Euler
 - *flock.render(window)* :Affichage SFML
 - *GUI* : Mise à jour sliders et lecture des valeurs

3. Interaction utilisateur :

- Sliders modifient *sepWeight*, *aliWeight*, *cohWeight*, *neighborRadius*
- Bouton "Apply" réinitialise la simulation avec les valeurs des sliders.

3.1.3 Organisation modulaire et extensibilité

Nous avons choisi cette architecture afin de faciliter les futures améliorations :

- **Ajout de comportements** : Nouvelles méthodes dans *Boid*

- **Optimisations spatiales** : Remplacement de *SpatialGrid* par un *Quadtree* par exemple
- **Parallélisation** :
 - OpenMP : Paralléliser la boucle dans *updateAll()*
 - MPI : Partitionner *Flock* en sous-domaines spatiaux
- **Passer en 3D** : Remplacer *Vector2D* par *Vector3D*, adapter *SpatialGrid* en 3D et utiliser OpenGL
- **GPU** : Migrer calculs de forces dans le GPU afin d'améliorer la répartition des tâches et avoir moins de calculs sur le processeur (possible gain de temps)

3.1.4 Pattern de conception utilisé

Nous avons choisi de séparer les tâches en quatre :

- Calcul -> Boid
- Stockage -> Flock
- Optimisation -> SpatialGrid
- Affichage -> GUI

La plus part des attributs sont publics afin de faciliter les appels de fonctions ou la récupération et la modification de valeurs.

Nous avons aussi choisi de ne pas utilisé de hiérarchie complexe d'héritage pour favoriser la flexibilité de notre code.

Enfin, les trois règles de Reynolds sont des stratégies de comportement modulables en utilisant les poids associés.

3.2 Modèle mathématique

Afin de représenter au mieux les différents comportements du banc de poisson, nous avons utilisé des formules mathématiques associées aux règles de flocking.

3.2.1 Règle 1 : Séparation

Le calcul de la séparation est nécessaire pour trouver notre vecteur de répulsion afin d'éviter les collisions. Nous avons utilisé la formule suivante :

$$\vec{s} = \frac{1}{N} \sum_{i=1}^N \frac{\vec{p} - \vec{p}_i}{|\vec{p} - \vec{p}_i|} \quad (1)$$

où

- $\vec{p}$: position du boid courant
- $\vec{p}_i$: position du voisin i
- N : nombre de voisins dans le rayon de perception
- La force de steering finale : $\vec{F}_s = (\hat{s} \cdot v_{max} - \vec{v})$ limité à F_{max} la force maximale

3.2.2 Règle 2 : Alignement

On calcul la vitesse moyenne des voisins afin de suivre la direction des voisins et pouvoir ajuster la vitesse. Nous avons utilisé la formule suivante :

$$\vec{a} = \frac{1}{N} \sum_{i=1}^N \vec{v}_i \quad (2)$$

où

- $\vec{v}_i$: vitesse du voisin i
- N : nombre de voisins
- Force de steering : $\vec{F}_a = (\hat{a} \cdot v_{max} - \vec{v})$ limité à F_{max} la force maximale

3.2.3 Règle 3 : Cohésion

On calcul le centre de masse des voisins afin de faire en sorte de diriger l'agent vers le centre de masse

$$\vec{c} = \frac{1}{N} \sum_{i=1}^N \vec{p}_i \quad (3)$$

où

- $\vec{t}$: position cible
- $\hat{x}$: vecteur normalisé

3.2.4 Paramètres clés

- `perceptionRadius` : rayon de détection des voisins
- `maxSpeed` : vitesse maximale du boid
- `maxForce` : force maximale applicable
- `separationWeight` : poids de la règle de séparation
- `alignmentWeight` : poids de la règle d'alignement
- `cohesionWeight` : poids de la règle de cohésion

3.3 Choix de conception

Afin de faciliter le développement, tout en gardant de bonnes performances et d'assurer la maintenabilité, nous avons pris différentes décisions que nous allons justifier.

3.3.1 Structure de données pour les boids

Pour les boids, nous avons choisi de les stocker directement dans une liste de pointeurs afin d'éviter les copies inutiles et coûteuses lors des différents appels dans les fonctions pour faire tourner la simulation.

3.3.2 Détection des voisins

Dans un premier temps, nous avons opté pour une approche naïve avec la méthode *getNeighbors()* qui effectue une recherche exhaustive parmi les boids, ce qui nous donne une complexité en $O(n^2)$. Cette première approche nous permet d'avoir une première vision sur les différents comportements. Par la suite, nous avons implémenté une structure *SpatialGrid* qui divise l'espace en cellules. Chaque boid ne regarde que ses cellules voisines. Cette implémentation nous permet de passer à une complexité de $O(n)$.

3.3.3 Gestion des bords

Actuellement, nous avons choisi de **téléporter** les boids du côté opposé de la zone de simulation lorsqu'il rentre en collision d'un bord. Nous avons fait ce choix afin de simplifier nos premières implémentation et avoir une simulation fonctionnelle.

Cependant, nous avons aussi pensé à des alternatives tels que le **rebond** où on aurait inverser la vitesse aux bords ou le **steering** où on aurait appliquer une force de répulsion quand le boid s'approche des bords.

3.3.4 Séparation modulaire Boid/Flock

Nous avons choisi de séparer les comportements individuels dans la classe *Boid* et la gestion collective dans la classe *Flock* afin de pouvoir tester chaque classe indépendamment, de pouvoir modifier l'algorithme de recherche de voisins sans toucher aux règles de Reynolds, de

pouvoir ajouter facilement de nouveaux comportements individuels et de pouvoir préparer la parallélisation future lors du semestre 2.

3.4 Implémentation

3.4.1 Structure de données

Dans notre implémentation, nous avons choisi quatre structures principales :

- **Vector2D** : Classe de base pour les opérations vectorielles 2D (position, vitesse, accélération)
- **Boid** : Agent autonome avec attributs physiques (mass, maxSpeed, maxForce, perceptionRadius)
- **Flock** : Gestionnaire d'agent autonome contenant le vecteur de boids et la SpatialGrid
- **SpatialGrid** : Grille d'accélération avec `std::vector<std::vector<Boid*>>`

3.4.2 Algorithmes principaux

Boucle de simulation :

Pour chaque boid:

1. Trouver les voisins (en utilisant SpatialGrid)
2. Calculer les forces de séparation, d'alignement et de cohésion
3. Appliquer les poids (en utilisant `applyForce` avec le total des forces)
4. Mettre à jour les positions (en utilisant `update()`)

SpatialGrid : Les boids sont repartis dans des cellules en fonction de leur position ce qui permet de regarder les 9 cellules adjacentes afin de trouver les positions des boids proche

3.4.3 Gestion de la mémoire

- Pas d'allocation dynamique manuelle de la mémoire (en utilisant `new`, `malloc` ou `delete`)
- Utilisation de conteneurs de la STL standards (`std::vector`)
- Réservation de capacité avec `reserve()` pour éviter les réallocations
- La grille est vidée et construite à chaque frame : `grid.clear()` puis `grid.addBoid()`

3.4.4 Optimisations appliquées

- **Surcharge d'opérateurs** pour Vector2D : Permet une syntaxe naturelle (`v1 + v2` ou `v * scalar`)
- **Utilisation de const** : Paramètres passés par référence constante quand nécessaire (`const Vector2D&`)
- **Utilisation de noexcept** : Utilisée sur les opérations vectorielles afin de laisser le compilateur faire les optimisations nécessaires
- **Normalisation optimisée** : Calcul de magnitude et de distance au carré afin d'éviter `sqrt()` inutiles
- **Flags de compilation** : `-O3 -march=native -flto` en mode Release

3.4.5 Contributions par membres

3.5 Tests et validation

3.5.1 Stratégie de test

Nous avons décidé de tester systématiquement l'intégralité de nos classes avec GoogleTest :

- `test_vector.bin` : Tests unitaires pour Vector2D

Membre	Contributions détaillées
Yohann	Vector 2D : Toutes les opérations vectorielles (addition, soustraction, multiplication, division), normalisation, magnitude, produit scalaire, rotation. Surchage complète des opérateurs. Tests unitaires des implémentations.
Iram	Boid : Implémentation des trois règles de Reynolds (separate, align, cohesion), méthodes <i>update()</i> , <i>applyForce()</i> , <i>seek()</i> , <i>flee()</i> , <i>getNeighbors()</i> . Tests unitaires pour Boid. GUI : Interface graphique avec sliders interactifs et affichage des FPS.
Laurence	Flock : Structure SpatialGrid complète, gestion de la collection de boids, boucle <i>updateAll()</i> , méthodes <i>populateRandom()</i> , <i>setWeights()</i> , <i>setNeighborRadius()</i> . Tests unitaires pour Flock.

TABLE 2 – Contributions détaillées par membre

- `test_boid.bin` : Tests unitaires pour Boid
- `test_flock.bin` : Tests unitaires pour Flock
- `test_ui.bin` : Tests d'intégration du GUI

Tout ces tests sont exécutés automatiquement grâce à la commande de CTest : `ctest -output-on-failure`.

3.5.2 Tests unitaires par classe

Vector2D (`test_vector.cpp`) :

- Opérations arithmétiques : addition, soustraction, multiplication, division
- Magnitude et `magnitudeSquared`
- Normalisation (avec cas limites tel que le vecteur nul)
- Produit scalaire (`dot product`)
- Distance entre deux vecteurs
- Rotation

Boid (`test_boid.cpp`) :

- Application de forces : *applyForce()* respecte la limite *maxForce*
- Mise à jour : *update()* limite la vitesse à *maxSpeed*
- Comportements : *seek()*, *flee()*, *separate()*, *align()* et *cohesion()*
- Détection de voisins : *getNeighbors()* dans le rayon *perceptionRadius*

Flock (`test_flock.cpp`) :

- Ajout/suppression de boids : *addBoid()*, *clear()*
- Population aléatoire : *populateRandom(N)*
- SpatialGrid : cohérence des cellules et recherche de voisins
- Mise à jour collective : *updateAll()* applique bien les règles

3.5.3 Validation des comportements émergents

En plus des tests unitaires automatisés, nous avons aussi effectué des validations visuelles :

- **Séparation** : Les boids évitent les collisions (pas de superposition prolongée)
- **Alignement** : Les boids se déplacent dans la même direction que son voisinage proche (ou au moins essaye de le faire en fonction de la vitesse de steering)
- **Cohésion** : Le banc reste groupé sans se disperser
- **Stabilité** : Pas d'explosions ou de vitesses aberrantes

— **Patterns émergents** : Formation de tourbillons, de vagues et de colonnes

3.5.4 Converture de tests

Toutes les méthodes publiques sont testées et les tests sont exécutés à chaque compilation afin de détecter les potentiels bugs pouvant apparaître pendant le développement. Les résultats de ces tests sont disponibles dans fichiers *build/Testing/Temporary*

3.6 Répartition des contributions

Le tableau 3 présente la repartition détaillée des contributions au projet.

Composant	Principal	Contributeurs
Vector2D	Yohann	Iram (intégration Boid)
Boid	Iram	Laurence (intégration Flock)
Flock	Laurence	Iram (logique simulation)
Benchmark	Laurence	-
GUI	Iram	-
Tests unitaire	Tous	Yohann (Vector2D), Iram (Boid), Laurence (Flock)
Documentation	Yohann	Tous
Rapport	Yohann	Tous (relecture)

TABLE 3 – Répartition récapitulative des contributions par composante

Statistiques Git :

- Total de commits : 82
- Iram : 38 commits
- Yohann : 36 commits
- Laurence : 8 commits

Tous les membres ont contribué de manière assez égale au projet, il faut cependant noter que Laurence n'a été ajouter au groupe que tardivement ce qui explique son plus petit nombre de commit et les commits réalisés sont plus gros.

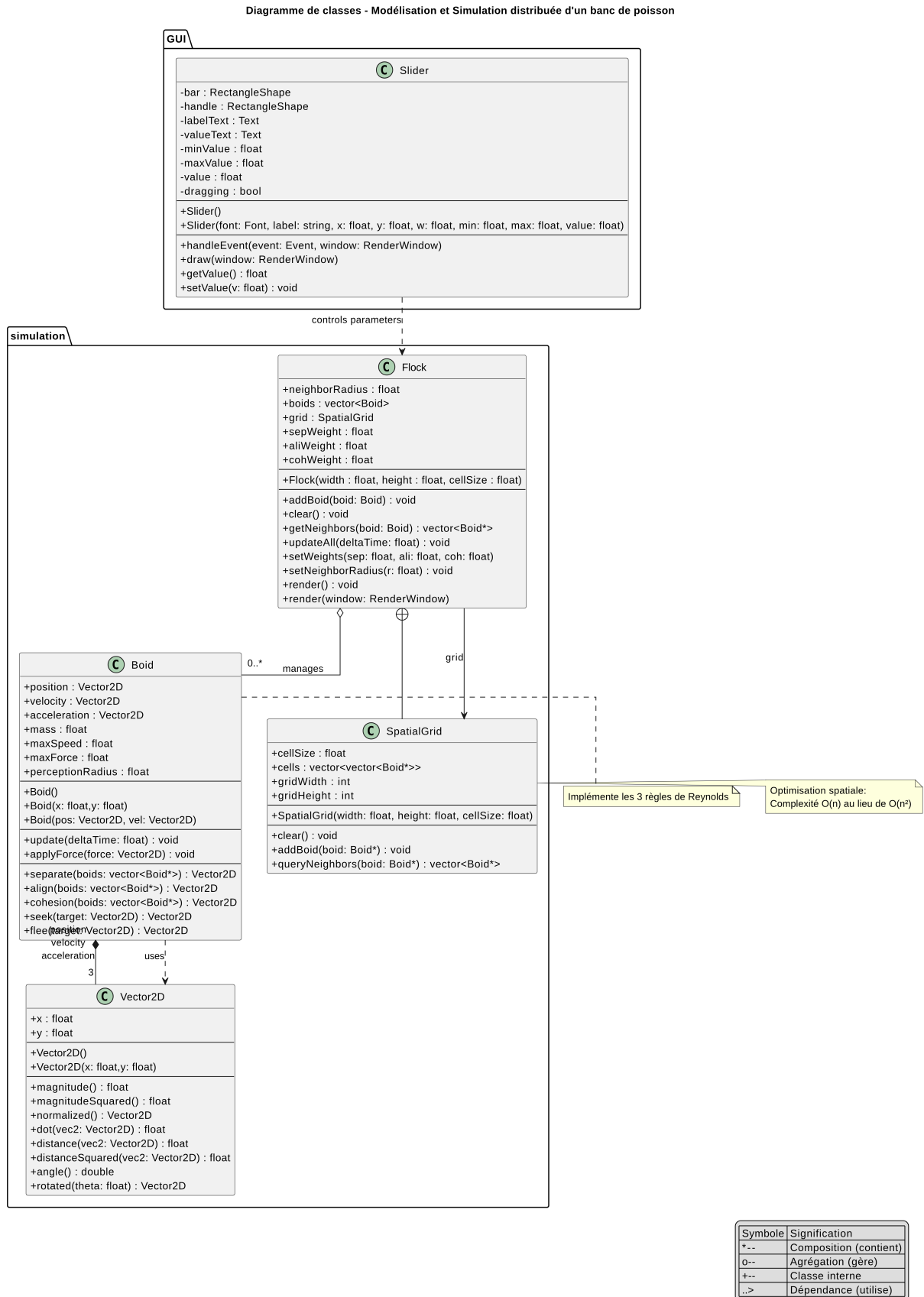


FIGURE 1 – Diagramme de classes UML du système de simulation